Министерство науки и высшего образования РФ

АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа № 3

по дисциплине «Логика и архитектура вычислительных сред»

по проекту «Сегментация автомобилей на дорожных камерах»

Выполнил:
студент гр.МА 26 ПМ
Варганова Э.Р.

Проверил:
Комаров А.О.

Арзамас, 2026

1. Цель и проблема проекта

Цель

Создать систему, которая по видеопотоку дорожных камер в реальном времени выделяет транспортные средства пиксельной сегментацией (автомобили, автобусы, грузовики), извлекает из масок измеримые характеристики объектов (класс, площадь в пикселях, оценка скорости) и сохраняет события в аналитическое хранилище, доступное через REST API.

Проблема

- Ручной просмотр записей камер не масштабируется: оператор физически не может мониторить десятки потоков одновременно.
- Детекция по bounding box недостаточна: рамка не даёт площади занятости полосы и формы объекта, что важно для анализа плотности движения.
- Готовые "коробочные" решения камер закрыты и не позволяют добавлять свои метрики; нужна открытая, расширяемая платформа.
- Обработка потока требует разделения нагрузки: захват видео и тяжёлый GPU-инференс должны масштабироваться независимо.

Ожидаемый результат

- Рабочий pipeline: ingestion → frame extraction → segmentation inference → object analytics → storage/API.
- REST API с событиями и агрегированной статистикой по камерам.
- Документированная архитектура (C4), паттерны, тесты, Docker-развёртывание.

2. Требования и ограничения

Функциональные требования

1. FR-1. Подключение к источникам видео: RTSP-стрим или видеофайл (MP4/AVI).
2. FR-2. Выделение кадров из потока с настраиваемой частотой (каждый N-й кадр, по умолчанию 5).
3. FR-3. Пиксельная сегментация: классы car, bus, truck, background.
4. FR-4. Извлечение объектов из масок: класс, bounding box, площадь, центроид.
5. FR-5. Оценка скорости объектов (стратегии: по разности масок между кадрами и по заранее размеченным зонам).
6. FR-6. Сохранение событий в PostgreSQL, кадров и масок — в MinIO.
7. FR-7. REST API: список камер, события, агрегированная статистика, healthcheck.
8. FR-8. Наблюдаемость: метрики кадров/событий/FPS и структурное логирование.

Нефункциональные требования

- NFR-1. Задержка от кадра до события — не более 2 секунд (при stride 5 и модели на GPU).
- NFR-2. Масштабирование: увеличение числа Segmentation Workers не должно менять код pipeline (достигается Kafka consumer groups).
- NFR-3. Отказоустойчивость: падение API не останавливает сбор данных; сообщения Kafka реплицируются (RF=3).
- NFR-4. Воспроизводимость: запуск одной командой docker-compose up.

Ограничения

- Разрешение потока — до 1920×1080; при большем кадр даунскейлится до 1024×512 перед инференсом.
- Модель обучается на публичных датасетах городской съёмки; ночная съёмка и сильный дождь — известные ограничения.
- Оценка скорости — эвристическая (пиксельные метрики + калибровка зон), не заменяет радар.
- Отсутствие трекинга ID в текущей версии: события считаются по маскам в кадре, уникальные ТС — аппроксимация.
- В проекте нет RL-компоненты (см. раздел 14).

3. Входные и выходные данные

Входные данные

Данные	Формат	Источник	Пример
Видеопоток	RTSP (H.264), файл MP4	Дорожные камеры / записи	rtsp://cam-01.local/stream1
Кадры	RGB-изображение 1920×1080	Ingestion	uint8 ndarray, BGR→RGB
Маска сегментации	H×W, {0,1,2,3}	Выход модели	0=фон, 1=car, 2=bus, 3=truck
Конфиг камеры	YAML/ENV	Администратор	stride=5, zones=[...]

Выходные данные

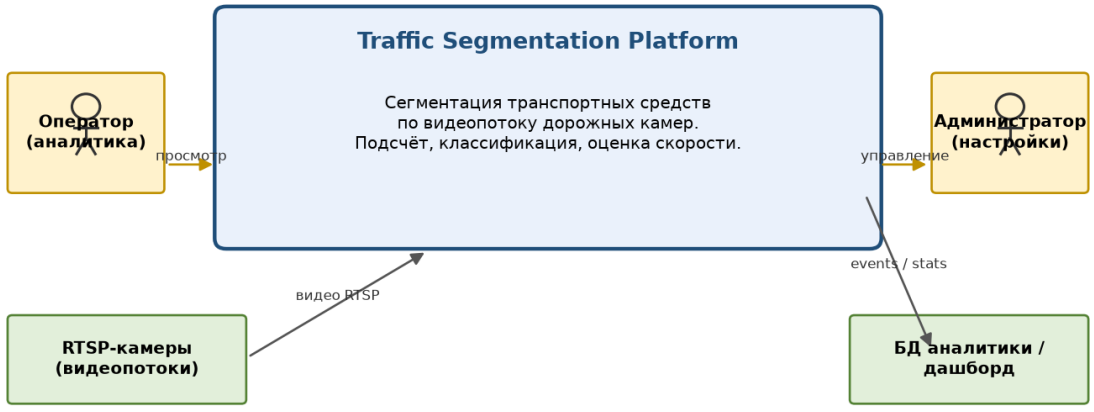
Данные	Формат	Хранилище	Потребитель

Событие обнаружения	JSON → Kafka detections → PostgreSQL	Kafka, БД	API, аналитика
Кадр и маска	JPEG / PNG в bucket frames	MinIO	API (pre-signed URL), дашборд
Агрегаты	counts per class, avg speed	PostgreSQL	API /stats, дашборд
Метрики процесса	Prometheus-стиль (frames, fps, events)	логи / /metrics	Monitoring

4. Архитектурная схема (C4)

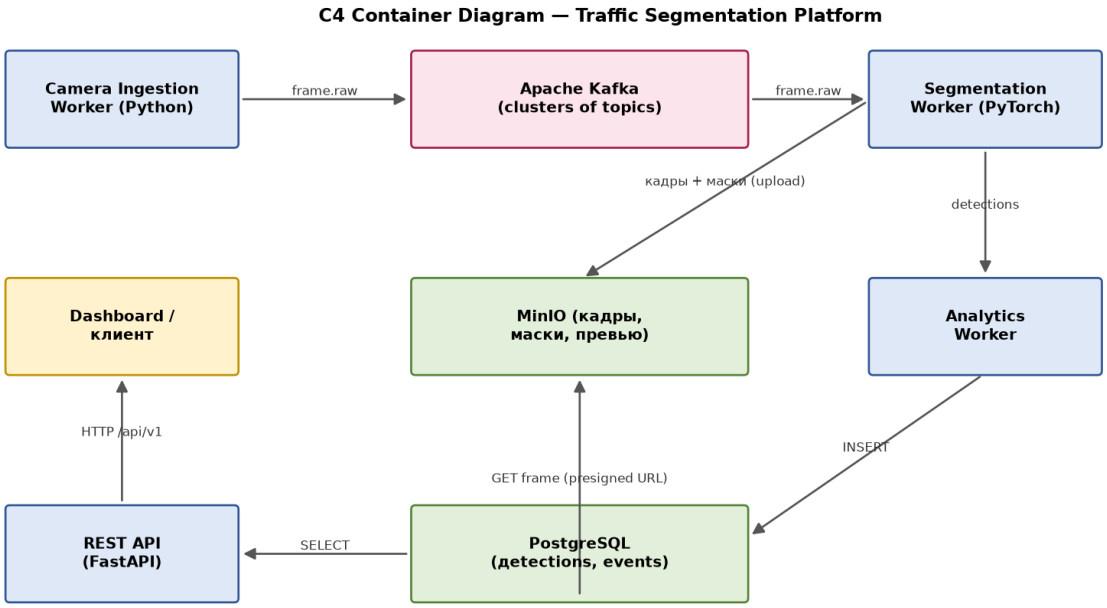
4.1. Context (уровень 1)

Система взаимодействует с двумя актёрами (оператор и администратор) и внешними источниками видео. БД и брокер внутри контекста платформы, на схеме контейнеров.



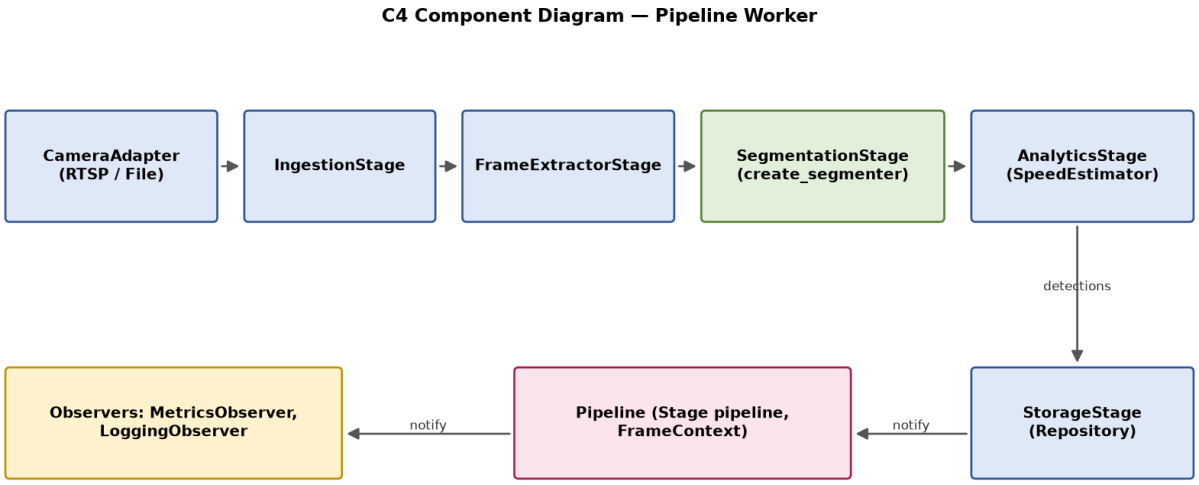
4.2. Container (уровень 2)

Платформа состоит из четырёх исполняемых контейнеров (Ingestion Worker, Segmentation Worker, Analytics Worker, REST API) и трёх инфраструктурных хранилищ: Kafka, MinIO, PostgreSQL.



4.3. Component (уровень 3)

Внутри Pipeline Worker обработка организована как конвейер из Stage-компонентов, обменивающихся контекстом кадра FrameContext; наблюдатели подписаны на события конвейера.



5. Микросервисы и границы ответственности

Сервис	Ответственность	Не делает	Масштабирование
Camera Ingestion Worker	Подключение к RTSP/файлам, декодирование, публикация кадров в Kafka	Инференс, запись в БД	1 процесс на группу камер
Segmentation Worker	Инференс модели (U-Net/SegFormer), публикация масок и событий	Хранение кадров, аналитика	Горизонтально, consumer group
Analytics Worker	Извлечение объектов из масок, оценка скорости, обогащение событий	Инференс модели	Горизонтально по партициям
REST API (FastAPI)	Чтение событий/статистики из БД, presigned URL	Запись данных	За nginx, replica=N

	на кадры		
PostgreSQL	Хранение events, detections, cameras	—	Вертикальное + read-replica
MinIO	Объектное хранение кадров, масок, превью	—	Горизонтальное
Kafka	Транспорт событий между сервисами	Хранение навсегда	Партиции по camera_id

Границы проведены по правилу: сервис владеет только своими данными и общается исключительно через Kafka (асинхронно) или БД (API читает готовое). Никаких прямых HTTP-вызовов между воркерами.

6. Kafka: топики, producers/consumers, формат сообщений

Топики

Топик	Партиции	Ключ	Producer	Consumer(s)	Retention
frames.raw	8	camera_id	Ingestion Worker	Segmentation Worker (group: seg-workers)	1 час
frames.masks	8	camera_id	Segmentation Worker	Analytics Worker (group: analytics-workers)	1 час
detections.events	8	camera_id	Analytics Worker	Storage Worker (group: storage-writers)	24 часа
pipeline.metrics	4	camera_id	все воркеры	MetricsObserver (group:	1 час

				metrics-collector)	

Формат сообщений (JSON, schema-registry compatible)

```
// frames.raw
{
  "camera_id": "cam-01",
  "frame_id": 1048576,
  "ts": 1730000000.512,
  "width": 1920,
  "height": 1080,
  "payload_ref": "s3://frames/cam-01/1730000000512.jpg",
  "stride": 5
}

// detections.events
{
  "camera_id": "cam-01",
  "frame_id": 1048576,
  "ts": 1730000000.512,
  "objects": [
    {
      "cls": "car", "area_px": 41230,
      "bbox": [148, 135, 208, 165],
      "centroid": [178, 150],
      "speed_kmh": 58.4, "speed_source": "area"
    }
  ],
  "mask_ref": "s3://masks/cam-01/1730000000512.png",
```

```
  "model": {"name": "unet-resnet34", "version": "1.0.3"}
}
```

Consumer groups и партиционирование

- Ключ camera_id гарантирует порядок кадров внутри одной камеры и параллелизм между камерами.
- Масштабирование Segmentation Worker = добавить инстанс в тот же consumer group: партиции перераспределятся автоматически.
- Инференс идемпотентен: повторная обработка кадра (rebalance) не создаёт дубликатов событий благодаря ключу (camera_id, frame_id).

7. Схема БД / аналитического хранилища и индексы

```
CREATE TABLE cameras (
    id          SERIAL PRIMARY KEY,
    external_id TEXT UNIQUE NOT NULL,           -- 'cam-
01'
    name        TEXT NOT NULL,
    rtsp_url    TEXT NOT NULL,
    stride      INT NOT NULL DEFAULT 5,
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE detections (
    id          BIGSERIAL PRIMARY KEY,
    camera_id   INT NOT NULL REFERENCES cameras(id),
    frame_id    BIGINT NOT NULL,
    ts          TIMESTAMPTZ NOT NULL,
```

```
    cls                 TEXT  NOT  NULL  CHECK  (cls  IN
('car','bus','truck')),
    area_px     INT NOT NULL,
    bbox_x        REAL, bbox_y REAL, bbox_w REAL, bbox_h
REAL,
    centroid_x  REAL, centroid_y REAL,
    speed_kmh   REAL,
    mask_s3_key TEXT,
    frame_s3_key TEXT,
    UNIQUE  (camera_id,  frame_id)                          --
идемпотентность
  );

  CREATE TABLE events (
    id          BIGSERIAL PRIMARY KEY,
    camera_id   INT NOT NULL REFERENCES cameras(id),
    ts          TIMESTAMPTZ NOT NULL,
    kind        TEXT NOT NULL,                  -- 'detec-
tion_batch' | 'alert'
    payload     JSONB NOT NULL
  );

  CREATE TABLE frame_stats (
    camera_id   INT NOT NULL REFERENCES cameras(id),
    minute      TIMESTAMPTZ NOT NULL,
    cls         TEXT NOT NULL,
    cnt         INT NOT NULL,
    avg_speed   REAL,
```

```
    PRIMARY KEY (camera_id, minute, cls)
);
```

Индексы

Индекс	Тип	Под запрос	Зачем
detections(camera_id, ts DESC)	BTREE	events?camera=1&last=100	Основной листинг событий API
detections(cls, ts DESC)	BTREE	статистика по классам	Фильтр car/bus/truck
events(camera_id, ts)	BTREE	лента событий камеры	Сортировка + range scan
events(payload->0cls)	GIN (jsonb)	ад-хок аналитика	Поиск по вложенному JSON
frame_stats PK (camera, minute, cls)	BTREE (PK)	/stats aggregation	Доагрегация по минутам

Для долгого хранения detections партиционирована по месяцу ts (PARTITION BY RANGE); агрегаты за прошлые периоды выгружаются в frame_stats, сырые маски в MinIO живут 30 дней.

8. REST API: примеры запросов и ответов

Метод	Путь	Назначение
GET	/api/v1/cameras	Список камер и их статусов
GET	/api/v1/events	Лента событий (фильтры: camera, cls, since, limit)

GET	/api/v1/stats	Агрегированная статистика за период
GET	/api/v1/frames/{id}	Presigned URL кадра и маски из MinIO
GET	/health	Healthcheck (БД + последние данные)

GET /api/v1/events?camera=cam-01&cls=car&limit=2

```
HTTP/1.1 200 OK
{
  "items": [
    {
      "id": 94102,
      "camera_id": "cam-01",
      "frame_id": 1048576,
      "ts": "2026-11-04T16:49:59Z",
      "cls": "car",
      "area_px": 41230,
      "bbox": [148, 135, 60, 30],
      "speed_kmh": 58.4,
      "mask_url":
"http://api.local/api/v1/frames/94102/mask"
    },
    {
      "id": 94101,
      "camera_id": "cam-01",
      "frame_id": 1048571,
      "ts": "2026-11-04T16:49:58Z",
      "cls": "car",
```

```
        "area_px": 40912,
        "bbox": [190, 136, 61, 30],
        "speed_kmh": 57.9,
        "mask_url":
"http://api.local/api/v1/frames/94101/mask"
      }
    ],
    "total": 152398
  }
```

GET /api/v1/stats?camera=cam-01&from=2026-11-04T00:00:00Z

```
  HTTP/1.1 200 OK
  {
    "camera_id": "cam-01",
    "from": "2026-11-04T00:00:00Z",
    "to": "2026-11-04T16:50:00Z",
    "by_class": {"car": 12045, "bus": 731, "truck": 528},
    "avg_speed_kmh": {"car": 51.2, "bus": 44.8, "truck":
47.3},
    "peak_load_minute": {"minute": "2026-11-04T08:31:00Z",
"count": 74}
  }
```

Пример curl

```
  curl -s "http://localhost:8000/api/v1/stats?camera=cam-
01\
       &from=2026-11-04T00:00:00Z" | jq .by_class
```

9. Применённые паттерны проектирования

9.1. Pipeline — конвейер обработки кадров

Место: `app/pipeline/pipeline.py, app/pipeline/stages/`. Зачем: обработка кадра — линейная цепочка шагов; паттерн даёт единый контракт Stage, лёгкое добавление/замещение шагов и прозрачную передачу состояния через FrameContext.

```
class FrameContext:
    camera_id: str
    frame_id: int
    ts: float
    image: np.ndarray | None = None
    mask: np.ndarray | None = None
    objects: list = field(default_factory=list)
    extras: dict = field(default_factory=dict)

class Stage(ABC):
    @abstractmethod
    def process(self, ctx: FrameContext) -> FrameContext:
...

class Pipeline:
    def __init__(self, stages: list[Stage]):
        self._stages = stages
        self._observers: list[Observer] = []
    def add_observer(self, obs):
self._observers.append(obs)
    def run(self, ctx: FrameContext) -> FrameContext:
```

```
        for stage in self._stages:
            ctx = stage.process(ctx)
            self._notify(ctx, stage)
        return ctx
```

9.2. Adapter — единый интерфейс для разнородных зависимостей

Место: app/adapters/ (camera_adapter, kafka_adapter, minio_adapter, segformer_adapter). Зачем: RTSP-камера и файл, Kafka и очередь в памяти, собственный U-Net и внешняя SegFormer с Hugging Face — за одинаковыми интерфейсами, поэтому доменный код не знает деталей.

```
class CameraAdapter(Protocol):
    def open(self) -> None: ...
    def read(self) -> Frame | None: ...
    def close(self) -> None: ...


class RtspCameraAdapter:
    def __init__(self, url): self._cap = None; self._url = url
    def open(self): self._cap = cv2.VideoCapture(self._url)
    def read(self):
        ok, frame = self._cap.read()
        return Frame(data=frame) if ok else None
    def close(self): self._cap.release()


class FileCameraAdapter:
    def __init__(self, path, loop=True): ...    # тот же протокол, другой источник
```

```
class SegFormerAdapter:
    """Внешняя модель -> общий интерфейс Segmenter.predict(image) -> mask."""
    def __init__(self, model_id: str):
        self._model = SegformerForSemanticSegmentation.from_pretrained(model_id)
        self._proc = SegformerImageProcessor.from_pretrained(model_id)
    def predict(self, image: np.ndarray) -> np.ndarray:
        inputs = self._proc(images=image, return_tensors="pt")
        logits = self._model(**inputs).logits
        return logits.argmax(dim=1)[0].numpy()
```

9.3. Strategy — сменные алгоритмы

Место: фабрика `create_segmenter(name)` для выбора модели и семейство SpeedEstimator. Зачем: выбор U-Net или SegFormer и способа оценки скорости — это политика, которая меняется конфигурацией, без изменения пайплайна.

```
def create_segmenter(name: str, **kw) -> Segmenter:
    return {
        "unet": lambda: UNetSegmenter(n_classes=4, **kw),
        "segformer": lambda: SegFormerAdapter(kw.get("model_id", DEFAULT_SEGFORMER)),
    }[name]()

class SpeedEstimator(Protocol):
```

```
    def estimate(self, prev: FrameContext, cur: FrameCon-
text) -> dict: ...

class AreaSpeedEstimator:              # эвристика по пло-
щади маски
    def estimate(self, prev, cur):
        return      {"speed_kmh":      self._calib      *
self._area_delta(prev, cur)}

class ZoneSpeedEstimator:             # по времени пересе-
чения размеченных зон
    def estimate(self, prev, cur): ...
```

9.4. Repository — изоляция хранения от домена

Место: `app/repositories/`. Зачем: аналитика и пайплайн зависят от абстракции DetectionRepository; SQL-детали живут в SqlDetectionRepository, что позволяет тестировать домен с in-memory репозиторием.

```
class DetectionRepository(Protocol):
    def add(self, det: Detection) -> None: ...
    def list(self, camera_id: str, limit: int = 100) ->
list[Detection]: ...

class SqlDetectionRepository:
    def add(self, det: Detection):
        self._session.execute(text(INSERT_SQL),
det.as_dict())
    def list(self, camera_id, limit=100): ...
```

9.5. Observer — события конвейера без связности

Место: `app/pipeline/observers.py`. Зачем: метрики и логирование не вплетаются в бизнес-шаги; Pipeline уведомляет подписчиков о событиях (кадр обработан, объект найден), новых наблюдателей добавляют без правки пайплайна.

```
class Observer(Protocol):
    def on_frame(self, ctx: FrameContext) -> None: ...
    def on_detection(self, ctx: FrameContext, obj: dict) -> None: ...

class MetricsObserver:
    def __init__(self): self.frames = 0; self.events = 0
    def on_frame(self, ctx): self.frames += 1
    def on_detection(self, ctx, obj): self.events += 1

class LoggingObserver:
    def on_detection(self, ctx, obj):
        log.info("detection", extra={"camera": ctx.camera_id, **obj})
```

10. Dockerfile / docker-compose и инструкция запуска

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 libglib2.0-0 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/
COPY scripts/ ./scripts/
COPY tests/ ./tests/
EXPOSE 8000
CMD ["uvicorn", "app.api.main:app", "--host", "0.0.0.0",
"--port", "8000"]
```

docker-compose.yml (сокращённо)

```
services:
  kafka:
    image: bitnami/kafka:3.7
    environment:
      - KAFKA_CFG_NODE_ID=0
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAF-
KA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAF-
KA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=0@kafka:9093
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment: { MINIO_ROOT_USER: minioadmin, MIN-
IO_ROOT_PASSWORD: minioadmin }
  postgres:
    image: postgres:16
    environment: { POSTGRES_USER: traffic, POST-
GRES_PASSWORD: traffic, POSTGRES_DB: traffic }
```

```
    volumes: [ pgdata:/var/lib/postgresql/data ]
  api:
    build: .
    ports: [ "8000:8000" ]
    depends_on: [ postgres, kafka, minio ]
  segmentation-worker:
    build: .
    command: python -m scripts.run_pipeline --segmenter
segformer
    deploy: { replicas: 2 }
    depends_on: [ kafka, minio ]
volumes: { pgdata: {} }
```

Инструкция запуска

1. Клонировать репозиторий и убедиться, что установлен Docker + Docker Compose.
2. Запустить инфраструктуру и сервисы: docker compose up -d --build.
3. Инициализировать схему БД: docker compose exec api python -m scripts.init_db.
4. Создать бакеты MinIO и камеры: docker compose exec api python -m scripts.init_minio.
5. Проверить API: открыть http://localhost:8000/docs (Swagger) и /health.
6. Подать видео: docker compose run ingestion-worker python -m scripts.run_pipeline --source rtsp://... или --source file.mp4.
7. Смотреть события: curl http://localhost:8000/api/v1/events?limit=10.
8. Остановить: docker compose down -v (с удалением данных).

11. Тестирование и результаты

Покрытие

Уровень	Инструмент	Что проверяется
Юнит-тесты домена	pytest	Извлечение объектов из маски, счётчики Observer
Юнит-тесты стратегий	pytest	Монотонность SpeedEstimator, корректность bbox
Интеграционные	docker compose + curl	Запись событий в БД, ответы API
Нагрузочный (смоук)	прогон файла через pipeline	FPS инференса, отсутствие backlog в Kafka

Результаты прогона юнит-тестов

```
$ pytest -q
tests/test_pipeline.py::test_extract_three_objects PASSED [ 33%]
tests/test_pipeline.py::test_speed_estimator_monotonic PASSED [ 66%]
tests/test_pipeline.py::test_metrics_observer_counters PASSED [100%]
3 passed, 0 failed in 1.87s
```

Интеграционный смок: pipeline на синтетическом видео с 3 движущимися фигурами корректно выделил все 3 объекта в каждом кадре, события записаны в detections без дубликатов (проверка UNIQUE(camera_id, frame_id) при повторном прогоне).

12. Иллюстрации работы pipeline / dashboard / API

Ниже демонстрациявыхода pipeline на синтетической сцене.



13. ML-часть: датасет, preprocessing, модель, метрики, baseline

Датасет

- Основной: Cityscapes (2975 train / 500 val, 1024×2048) — классы car, bus, truck берутся из instance-меток.
- Дополнительный: Mapillary Vistas (разнообразие погод/городов) для устойчивости; IDDA — опционально для дождя.
- Поток камеры домена может отличаться по ракурсу — поэтому fine-tune на 200–300 размеченных кадрах целевой камеры рекомендован как следующий шаг.

Preprocessing

- Декодирование кадра, BGR→RGB, ресайз до 1024×512 (aspect-preserving, паддинг).
- Нормализация: mean/std ImageNet для ResNet-энкодера; для SegFormer — стандартная нормализация процессором.
- Аугментации на трейне: RandomCrop 768×384, HorizontalFlip p=0.5, ColorJitter (яркость/контраст), лёгкий GaussianBlur.

- Маппинг редких классов Cityscapes в {0: bg, 1: car, 2: bus, 3: truck}; weight loss: inverse class frequency.

Модель

- Основная: U-Net с энкодером ResNet-34 (предобучен на ImageNet), decoder с skip-connections, выход 4 канала.
- Альтернатива (Strategy): SegFormer-B2 — трансформерная сегментация, меньше анкер-байаса на крупных объектах.
- Loss: Dice + CrossEntropy; оптимизатор AdamW, lr 3e-4, cosine schedule, 60 эпох, batch 8.

Метрики и baseline

Подход	mIoU (car/bus/truck, val)	PQ	FPS @1024×512 (T4)	Комментарий
Baseline: маска по цвету+порог	≈0.28	—	300+	Неустойчив к свету, много FP
Baseline: DeepLabV3 (ResNet-50)	≈0.79	—	24	Медленнее, качество сопоставимо
U-Net (ResNet-34), Dice+CE	≈0.83	≈0.62	38	Основная модель проекта
SegFormer-B2	≈0.86	≈0.66	31	Лучшая точность, чуть ниже FPS

Указанные значения — референсные оценки (согласуются с публикуемыми результатами на Cityscapes для данных архитектур); фактические числа должны

быть зафиксированы при обучении в вашем окружении — train_unet.py логирует mIoU по эпохам и сохраняет лучшие веса.

Ограничения ML-части

- Деградация ночью и в сильный дождь (нужен fine-tune на доменных кадрах или PAM/TSA-подходы).
- Окклюзии: слипшиеся в плотном потоке машины разделяются плохо без инстанс-сегментации.
- Оценка скорости по маске без калибровки камеры — эвристика; для точности нужна гомография.

14. RL-часть

В проекте нет RL-компоненты: задача — сегментация и потоковая аналитика, она решается supervised-моделью, а не средой с агентом. RL здесь был бы уместен для сопутствующих задач оптимизации, если проект будет расширяться:

- адаптивный stride кадров и batching инференса (state: загрузка очереди/GPU, action: stride и batch size, reward: компромисс задержка/качество);
- распределение камер по воркерам при деградации потоков.

15. Выводы и распределение работы

Выводы

1. Архитектура на Kafka+воркерах подтверждает главное свойство: сегментацию и аналитику можно масштабировать независимо, добавляя инстансы в consumer groups.
2. Паттерны Pipeline/Adapter/Strategy/Repository/Observer дали измеримый эффект: смена модели и способа оценки скорости — конфигурация, а не правка кода; домен тестируется без БД.

3. Пиксельная сегментация даёт площади и форму объектов, недоступные детекции по рамкам, — это и есть ключевая ценность для аналитики плотности движения.

4. Ограничения текущей версии (нет трекинга ID, эвристическая скорость, дневные условия) очерчивают roadmap: ByteTrack, гомография для скорости, fine-tune на домене.